☕ Métodos e Vetores em Java

🎯 Objetivos de Aprendizagem

Ao final deste material, espera-se que você seja capaz de:

- Compreender o conceito de **métodos** em Java;
- Criar métodos com e sem parâmetros;
- Criar métodos com e sem retorno;
- Compreender a utilização da instrução `return`;
- Compreender o conceito de **vetores (arrays)**;
- Declarar, criar e inicializar vetores;
- Acessar elementos por meio de seus índices;
- Percorrer vetores utilizando o `for` tradicional;
- Utilizar o **Enhanced for** ou **for-each**;
- Utilizar métodos para manipular vetores;
- Organizar programas dividindo responsabilidades entre métodos.

1. Métodos em Java

1.1 O que é um método?

Um **método** é um bloco de código responsável por realizar uma determinada tarefa dentro de um programa.

Em vez de colocar todas as instruções dentro do método `main`, podemos dividir o programa em partes menores. Cada parte pode possuir uma responsabilidade específica.

> 💡 **Pense da seguinte maneira:**
> Se um programa fosse uma empresa, cada método poderia ser comparado a um funcionário especializado em executar uma determinada tarefa.

Exemplo:

```
public class Exemplo {

    public static void main(String[] args) {

        exibirMensagem();

    }

    public static void exibirMensagem() {

        System.out.println("Olá, Java!");

    }

}
```

Quando escrevemos:

```
exibirMensagem();
```

estamos **chamando** ou **invocando** o método.

1.2 Estrutura de um método

De forma simplificada, um método possui a seguinte estrutura:

```
modificador tipoRetorno nomeMetodo(parâmetros) {

    // instruções

}
```

Por exemplo:

```
public static int somar(int numero1, int numero2) {

    int resultado = numero1 + numero2;

    return resultado;

}
```

Elemento	Descrição
`public`	Modificador de acesso.
`static`	Permite chamar o método diretamente a partir do método `main`, que também é estático.
`int`	Tipo de dado retornado pelo método.
`somar`	Nome atribuído ao método.
`numero1, numero2`	Parâmetros recebidos pelo método.
`return`	Retorna um valor para o local que realizou a chamada do método.

1.3 Método sem retorno — `void`

Quando um método executa uma tarefa, mas não precisa devolver nenhum resultado, utilizamos a palavra `void`.

```
public static void exibirBoasVindas() {

    System.out.println("Bem-vindo ao curso de Java!");

}
```

Para executar esse método:

```
public static void main(String[] args) {

    exibirBoasVindas();

}
```

1.4 Métodos com parâmetros

Parâmetros permitem enviar informações para um método. Dessa forma, um mesmo método pode trabalhar com diferentes valores.

```
public static void exibirNome(String nome) {

    System.out.println("Olá, " + nome);

}
```

Chamando o método:

```
public static void main(String[] args) {

    exibirNome("Carlos");
    exibirNome("Mariana");
    exibirNome("João");

}
```

Saída:

```
Olá, Carlos
Olá, Mariana
Olá, João
```

1.5 Métodos que retornam valores

Um método pode realizar um processamento e **devolver um resultado** para quem realizou sua chamada.

Nesse caso, devemos informar o tipo do valor que será retornado.

```
public static double calcularMedia(double nota1, double nota2) {

    double media = (nota1 + nota2) / 2;

    return media;

}
```

Podemos armazenar o resultado retornado pelo método em uma variável:

```
public static void main(String[] args) {

    double resultado = calcularMedia(8.0, 7.0);

    System.out.println("Média: " + resultado);

}
```

Saída:

```
Média: 7.5
```

1.6 Exemplo completo utilizando métodos



```
public class Calculadora {

    public static void main(String[] args) {

        int numero1 = 10;
        int numero2 = 5;

        int resultadoSoma = somar(numero1, numero2);

        int resultadoSubtracao = subtrair(numero1, numero2);

        System.out.println("Soma: " + resultadoSoma);
        System.out.println("Subtração: " + resultadoSubtracao);

    }

    public static int somar(int numero1, int numero2) {

        return numero1 + numero2;

    }

    public static int subtrair(int numero1, int numero2) {

        return numero1 - numero2;

    }

}
```

✅ Boa prática

Um método deve possuir uma **responsabilidade bem definida**.

Por exemplo:

```
calcularMedia()
buscarAluno()
calcularDesconto()
exibirRelatorio()
```

Apenas pelo nome conseguimos compreender a responsabilidade de cada método.

2. Vetores em Java

2.1 O problema das várias variáveis

Imagine que precisamos armazenar cinco notas.

Poderíamos criar cinco variáveis:

```
double nota1 = 8.0;
double nota2 = 7.5;
double nota3 = 9.0;
double nota4 = 6.0;
double nota5 = 8.5;
```

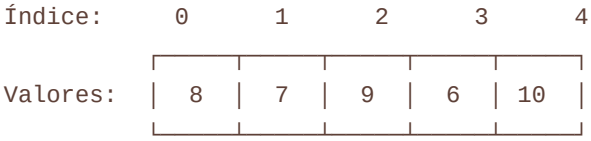
Para poucos valores isso pode funcionar. Entretanto, imagine armazenar **100, 500 ou 10.000 valores**.

Para resolver esse tipo de problema podemos utilizar uma estrutura chamada **vetor**.

2.2 O que é um vetor?

Um **vetor**, também chamado de **array**, é uma estrutura que permite armazenar vários valores do **mesmo tipo** utilizando uma única variável.

Imagine o vetor como uma sequência de caixas:



Em Java, o primeiro elemento de um vetor possui índice **0**.

2.3 Declarando um vetor

A sintaxe básica para declaração de um vetor é:

```
tipo[] nomeVetor;
```

Exemplos:

```
int[] numeros;

double[] notas;

String[] nomes;
```

2.4 Criando um vetor

Após declarar o vetor, precisamos definir a quantidade de posições que ele possuirá.

```
int[] numeros = new int[5];
```

Nesse exemplo, o vetor possui espaço para armazenar **cinco números inteiros**.

```
Índices:

0   1   2   3   4
```

⚠ **Atenção:**
Um vetor criado com tamanho 5 possui índices de **0 até 4**.

Portanto:

```
numeros[5]
```

tentará acessar uma posição que não existe.

2.5 Inserindo valores no vetor

Utilizamos o índice para indicar a posição em que um valor será armazenado.

```
int[] numeros = new int[5];

numeros[0] = 10;
numeros[1] = 20;
numeros[2] = 30;
numeros[3] = 40;
numeros[4] = 50;
```

Após essas atribuições:

```
Índice:     0      1      2      3      4
          ┌──────┬──────┬──────┬──────┬──────┐
Valores:  | 10   | 20   | 30   | 40   | 50   |
          └──────┴──────┴──────┴──────┴──────┘
```

2.6 Inicialização direta

Também podemos criar um vetor já informando seus valores.

```
int[] numeros = {10, 20, 30, 40, 50};
```

Nesse caso, Java identifica automaticamente que o vetor possui **cinco posições**.

2.7 Acessando elementos

Para acessar um elemento, informamos o nome do vetor e o índice desejado.

```
int[] numeros = {10, 20, 30, 40, 50};

System.out.println(numeros[0]);
System.out.println(numeros[2]);
System.out.println(numeros[4]);
```

Saída:

```
10
30
50
```

2.8 Descobrindo o tamanho — `length`

Todo vetor possui a propriedade `length`, que informa sua quantidade de posições.

```
int[] numeros = {10, 20, 30, 40, 50};

System.out.println(numeros.length);
```

Resultado:

```
5
```

💡 **Importante:**
Para vetores utilizamos:

```
vetor.length
```

Observe que `length` não possui parênteses.

2.9 Percorrendo um vetor com `for`

Uma das maneiras mais utilizadas para percorrer um vetor é utilizar uma estrutura de repetição `for`.

```
int[] numeros = {10, 20, 30, 40, 50};

for (int i = 0; i < numeros.length; i++) {

    System.out.println(numeros[i]);

}
```

A variável `i` representa o índice do elemento que está sendo acessado.

💡 **Observe:**

```
i = 0 → numeros[0]
i = 1 → numeros[1]
i = 2 → numeros[2]
i = 3 → numeros[3]
i = 4 → numeros[4]
```

2.10 Enhanced `for` — `for-each`

Java possui uma forma simplificada de percorrer todos os elementos de um vetor chamada **Enhanced for**, também conhecida como **for-each**.

💡 **Importante:**
Em Java, **Enhanced for** e **for-each** são dois nomes utilizados para se referir à **mesma estrutura de repetição**.

Diferentemente do `for` tradicional, o Enhanced `for` não trabalha diretamente com o índice do vetor.

A cada repetição, uma variável recebe automaticamente um dos elementos armazenados.

Sintaxe

```
for (tipo variavel : vetor) {

    // instruções

}
```

Podemos interpretar como: "Para cada elemento existente no vetor, execute as instruções."

Exemplo com números inteiros



```
int[] numeros = {10, 20, 30, 40, 50};

for (int numero : numeros) {

    System.out.println(numero);

}
```

A variável `numero` recebe um elemento diferente do vetor a cada repetição.

Funcionamento:

```
1ª repetição → numero = 10
2ª repetição → numero = 20
3ª repetição → numero = 30
4ª repetição → numero = 40
5ª repetição → numero = 50
```

Saída:

```
10
20
30
40
50
```

Comparando `for` tradicional e Enhanced `for`

Considere o vetor:

```
int[] numeros = {10, 20, 30, 40, 50};
```

Utilizando o `for` tradicional:

```
for (int i = 0; i < numeros.length; i++) {

    System.out.println(numeros[i]);

}
```

Nesse caso, `i` representa o **índice**.

Utilizando Enhanced `for` / `for-each`:

```
for (int numero : numeros) {

    System.out.println(numero);

}
```

Nesse caso, `numero` representa diretamente **o valor armazenado no vetor**.

Característica	for tradicional	Enhanced for / for-each
Trabalha diretamente com índice	Sim	Não
Acesso ao elemento	`vetor[i]`	Variável da repetição

Quantidade de código	Maior	Menor
Uso recomendado	Quando precisamos trabalhar com as posições	Quando precisamos apenas percorrer os elementos

Exemplo com vetor de `String`

O Enhanced `for` também pode ser utilizado com outros tipos de dados.

```
String[] alunos = {
    "Ana",
    "Carlos",
    "Mariana",
    "Pedro"
};

for (String aluno : alunos) {

    System.out.println(aluno);

}
```

Saída:

```
Ana
Carlos
Mariana
Pedro
```

Somando valores com Enhanced `for`

```
int[] numeros = {10, 20, 30, 40, 50};

int soma = 0;

for (int numero : numeros) {

    soma += numero;

}

System.out.println("Soma: " + soma);
```

Resultado:

```
Soma: 150
```

Calculando a média com Enhanced `for`

```
double[] notas = {7.5, 8.0, 6.5, 9.0};

double soma = 0;

for (double nota : notas) {

    soma += nota;

}

double media = soma / notas.length;

stem.out.println("Média: " + media);
```

⚠ Quando utilizar o for tradicional?

O Enhanced `for` é excelente quando queremos apenas percorrer os valores.

Entretanto, existem situações em que precisamos saber exatamente **qual posição está sendo acessada**.

Por exemplo:

```
for (int i = 0; i < numeros.length; i++) {

    System.out.println(
        "Posição " + i + ": " + numeros[i]
    );

}
```

Nesse caso, o `for` tradicional é mais adequado porque precisamos utilizar o índice `i`.

2.11 Preenchendo um vetor pelo teclado

Podemos combinar `Scanner`, vetores e uma estrutura de repetição para solicitar vários valores ao usuário.

```
import java.util.Scanner;

public class Exemplo {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        int[] numeros = new int[5];

        for (int i = 0; i < numeros.length; i++) {

            System.out.print(
                "Digite o número " + (i + 1) + ": "
            );

            numeros[i] = scanner.nextInt();

        }

        System.out.println("\nValores informados:");

        for (int numero : numeros) {

            System.out.println(numero);

        }

        scanner.close();

    }

}
```

🔎 **Observe:**
O `for` tradicional foi utilizado para **preencher** as posições do vetor.

O Enhanced `for` foi utilizado para **percorrer e exibir** os valores armazenados.

2.12 Encontrando o maior valor

Para encontrar o maior elemento, podemos inicialmente considerar que o primeiro valor do vetor é o maior.

```
int[] numeros = {15, 8, 27, 4, 19};

int maior = numeros[0];

for (int i = 1; i < numeros.length; i++) {

    if (numeros[i] > maior) {

        maior = numeros[i];

    }

}

System.out.println("Maior valor: " + maior);
```

Resultado:

```
Maior valor: 27
```

3. Utilizando Métodos com Vetores

Métodos e vetores podem ser utilizados em conjunto. Um vetor pode ser enviado como parâmetro para que um método realize algum processamento.

3.1 Passando um vetor para um método

```
public static void exibirVetor(int[] numeros) {

    for (int numero : numeros) {

        System.out.println(numero);

    }

}
```

Para utilizar o método:

```
int[] valores = {5, 10, 15, 20};

exibirVetor(valores);
```

3.2 Método para somar os valores

O vetor pode ser recebido como parâmetro e o método pode retornar o resultado do processamento.

```
public static int calcularSoma(int[] numeros) {

    int soma = 0;

    for (int numero : numeros) {

        soma += numero;

    }

    return soma;
```

3.3 Método para calcular a média

```
public static double calcularMedia(double[] notas) {

    double soma = 0;

    for (double nota : notas) {

        soma += nota;

    }

    return soma / notas.length;

}
```

3.4 Utilizando Enhanced `for` em Métodos

O Enhanced `for` é especialmente útil dentro de métodos que precisam percorrer todos os valores de um vetor.

Método para exibir números

```
public static void exibirNumeros(int[] numeros) {

    for (int numero : numeros) {

        System.out.println(numero);

    }

}
```

Método para calcular a soma

```
public static int calcularSoma(int[] numeros) {

    int soma = 0;

    for (int numero : numeros) {

        soma += numero;

    }

    return soma;

}
```

3.5 Exemplo completo utilizando Métodos + Vetores + Enhanced for

```
public class ExemploVetor {

    public static void main(String[] args) {

        int[] numeros = {10, 20, 30, 40, 50};

        exibirNumeros(numeros);

        int soma = calcularSoma(numeros);

        System.out.println("Soma: " + soma);

    }


    public static void exibirNumeros(int[] numeros) {

        for (int numero : numeros) {

            System.out.println(numero);

        }

    }


    public static int calcularSoma(int[] numeros) {

        int soma = 0;

        for (int numero : numeros) {

            soma += numero;

        }

        return soma;

    }

}
```

4. Erros Comuns ao Trabalhar com Vetores

❌ Erro 1 — Acessar uma posição inexistente

```
int[] numeros = new int[5];

numeros[5] = 10;
```

Um vetor de tamanho 5 possui as posições:

```
0, 1, 2, 3 e 4
```

Tentar acessar a posição 5 provoca um erro em tempo de execução:

```
ArrayIndexOutOfBoundsException
```

❌ Erro 2 — Utilizar <= ao percorrer o vetor

Incorreto:



```
for (int i = 0; i <= numeros.length; i++) {

    System.out.println(numeros[i]);

}
```

Quando `i` for igual a `numeros.length`, o programa tentará acessar uma posição inexistente.

Correto:

```
for (int i = 0; i < numeros.length; i++) {

    System.out.println(numeros[i]);

}
```

5. O Tamanho de um Vetor Pode Aumentar?

Não. Depois que um vetor é criado em Java, seu tamanho é **fixo**.

```
int[] numeros = new int[5];
```

Esse vetor possuirá exatamente **cinco posições** durante toda a sua existência.

> 💡 **Importante:**
> Caso seja necessário armazenar uma quantidade maior de elementos, será necessário criar outro vetor com uma capacidade maior.

5.1 Criando um novo vetor maior

```
int[] numeros = {10, 20, 30};

int[] novoVetor = new int[5];

for (int i = 0; i < numeros.length; i++) {

    novoVetor[i] = numeros[i];

}

novoVetor[3] = 40;
novoVetor[4] = 50;
```

Nesse exemplo, os elementos do vetor original foram copiados para um novo vetor com maior capacidade.

6. Exemplo Integrador — Sistema de Notas

Considere a seguinte situação:

Uma turma possui cinco alunos. O programa deverá:

1. Solicitar as cinco notas;
2. Armazenar as notas em um vetor;
3. Calcular a média da turma;

4. Encontrar a maior nota;
5. Exibir todas as notas;
6. Exibir a média e a maior nota.

Implementação



```
import java.util.Scanner;

public class Turma {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        double[] notas = new double[5];

        preencherNotas(scanner, notas);

        double media = calcularMedia(notas);

        double maiorNota = encontrarMaior(notas);

        exibirNotas(notas);

        System.out.printf(
            "%nMédia da turma: %.2f%n",
            media
        );

        System.out.printf(
            "Maior nota: %.2f%n",
            maiorNota
        );

        scanner.close();

    }


    public static void preencherNotas(
            Scanner scanner,
            double[] notas) {

        for (int i = 0; i < notas.length; i++) {

            System.out.print(
                "Digite a nota do aluno "
                + (i + 1)
                + ": "
            );

            notas[i] = scanner.nextDouble();

        }

    }


    public static double calcularMedia(double[] notas) {

        double soma = 0;

        for (double nota : notas) {

            soma += nota;

        }

        return soma / notas.length;

    }


    public static double encontrarMaior(double[] notas) {

        double maior = notas[0];

        for (double nota : notas) {

            if (nota > maior) {
```

```
                maior = nota;
            }
        }
        return maior;
    }

    public static void exibirNotas(double[] notas) {
        System.out.println("\nNotas cadastradas:");
        for (double nota : notas) {
            System.out.println(nota);
        }
    }
}
```

🔎 Organização do programa

```
main()
   │
   ├── preencherNotas()
   │
   ├── calcularMedia()
   │
   ├── encontrarMaior()
   │
   └── exibirNotas()
```

O método `main` coordena a execução do programa, enquanto os demais métodos possuem responsabilidades específicas.

✅ Conceitos aplicados neste exemplo

- Criação de vetor;
- Leitura de valores utilizando `Scanner`;
- `for` tradicional;
- Enhanced `for` / `for-each`;
- Passagem de vetor como parâmetro;
- Métodos com parâmetros;
- Métodos com retorno;
- Métodos sem retorno;
- Uso de `return`;
- Uso da propriedade `length`;
- Separação de responsabilidades entre métodos.

7 Quando Usar Cada Tipo de `for`?

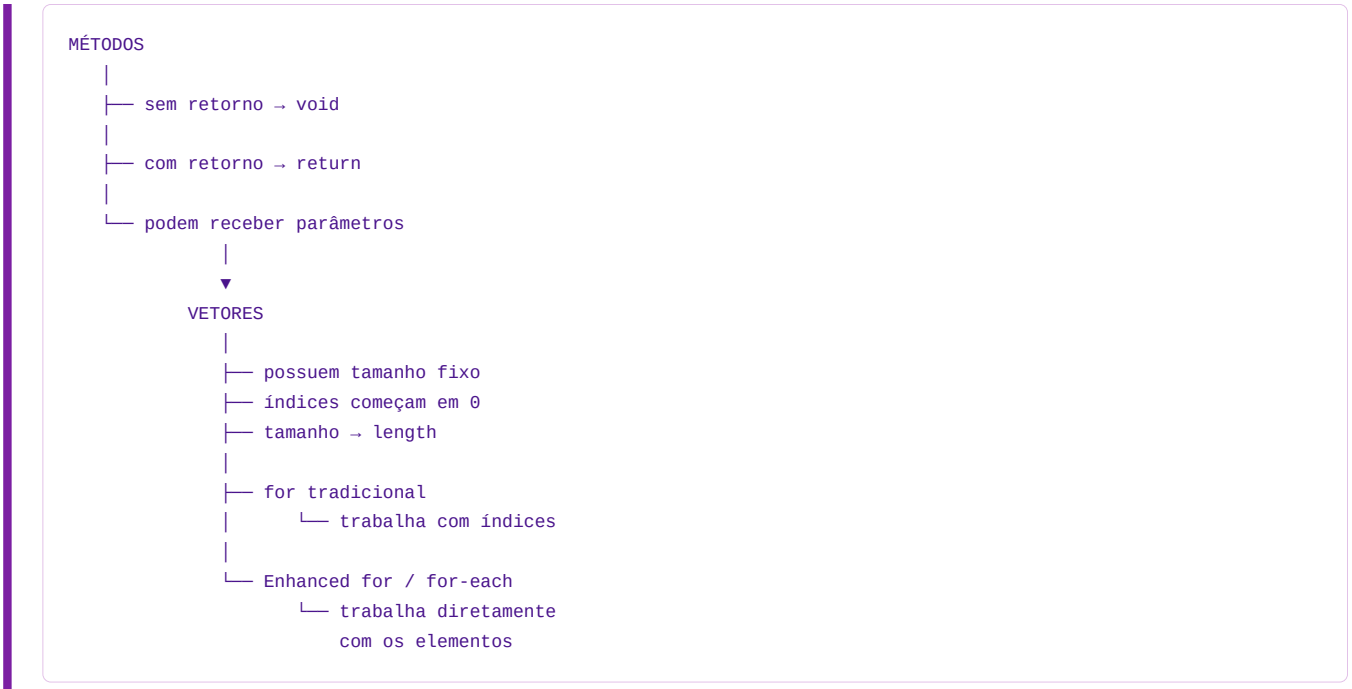
Situação	Recomendado
Preciso saber a posição do elemento	`for` tradicional
Preciso preencher determinadas posições	`for` tradicional
Preciso exibir todos os elementos	Enhanced `for` / `for-each`
Preciso somar todos os valores	Enhanced `for` / `for-each`
Preciso calcular uma média	Enhanced `for` / `for-each`
Preciso acessar elementos em uma ordem específica	`for` tradicional

8. Resumo

Conceito	Exemplo
Método sem retorno	`void exibir()`
Método com retorno	`int somar()`
Método com parâmetro	`somar(int a, int b)`
Retorno	`return resultado;`
Declaração de vetor	`int[] numeros;`
Criação de vetor	`new int[10]`
Inicialização direta	`int[] numeros = {10, 20, 30};`
Acessar um elemento	`numeros[0]`
Tamanho do vetor	`numeros.length`
Percorrer utilizando índice	`for (int i = 0; i < vetor.length; i++)`
Enhanced for / for-each	`for (int valor : vetor)`
Vetor como parâmetro	`calcularMedia(double[] notas)`

🧠 Para lembrar





☕ Java

Métodos ajudam a organizar as responsabilidades do programa.

Vetores permitem organizar conjuntos de dados do mesmo tipo.

O **for** tradicional trabalha muito bem com índices, enquanto o **Enhanced for / for-each** facilita a leitura e o processamento dos elementos.

Organizar o código é uma parte fundamental da programação.

Última atualização: sexta-feira, 21 ago. 2026, 13:59